

Policy Gradient Methods for Continuous Decision Processes

Om Abhijit Shishupal

Self Project

July 2025

Abstract

We present a comprehensive study of policy gradient reinforcement learning for continuous control problems formulated as Markov Decision Processes (MDPs). Starting from first principles, we derive the policy gradient theorem, implement stochastic Gaussian policies parameterised by deep neural networks, and develop several gradient estimators including REINFORCE (williams1992), Advantage Actor-Critic (A2C), and the Natural Policy Gradient (kakade2001). A central focus is variance reduction: we empirically study the effect of baseline subtraction on gradient estimator signal-to-noise ratio and show that neural network value baselines reduce variance by up to 60% compared to the no-baseline estimator. We extend the framework to continuous action spaces via Gaussian policies and apply it to linear-quadratic regulator (LQR) control and nonlinear pendulum swing-up tasks. Finally, we establish and experimentally verify the connection between policy gradient RL and stochastic optimal control by comparing learned value functions against the analytic Hamilton–Jacobi–Bellman (HJB) solution obtained through the Discrete Algebraic Riccati Equation. Our implementation achieves less than 5% regret relative to the analytic optimal LQR controller on a 4-dimensional system, and successfully learns to swing up and balance a continuous pendulum without access to system dynamics.

Contents

1	Introduction	3
2	Markov Decision Processes	3
2.1	Policies and Trajectories	4
2.2	The Policy Optimisation Objective	4
3	The Policy Gradient Theorem	4
3.1	Derivation	4
3.2	The Score Function	5
3.3	The REINFORCE Estimator	5
4	Variance Reduction	5
4.1	The Variance Problem	5
4.2	Baseline Subtraction	6
4.3	Advantage Estimation	6
5	Algorithms	7
5.1	REINFORCE	7
5.2	Advantage Actor-Critic (A2C)	7
5.3	Natural Policy Gradient	7

6	Gaussian Policies for Continuous Action Spaces	8
6.1	Parameterisation	8
6.2	Score Function for Gaussian Policy	8
6.3	Reparameterisation Trick	8
6.4	Network Architecture	9
7	Connection to Stochastic Optimal Control	9
7.1	The Hamilton-Jacobi-Bellman Equation	9
7.2	Linear-Quadratic Regulator	9
7.3	RL as Approximate Dynamic Programming	9
8	Experiments	10
8.1	Experimental Setup	10
8.2	Experiment 1: LQR Policy Gradient vs Analytic Optimum	10
8.3	Experiment 2: Continuous Pendulum Swing-Up	11
8.4	Experiment 3: Variance Reduction Study	11
8.5	Experiment 4: HJB Connection	11
9	Discussion	12
9.1	Comparative Analysis of Algorithms	12
9.2	Gaussian Policies and Exploration	12
9.3	RL vs Classical Control	12
10	Conclusion	12
A	Proof of Variance Reduction Bound	13
B	Discrete Algebraic Riccati Equation	13
C	Experimental Hyperparameters	13

1 Introduction

Sequential decision-making under uncertainty is a fundamental challenge in engineering, robotics, and operations research. The reinforcement learning (RL) paradigm formalises this as an agent interacting with an environment modelled as a Markov Decision Process (MDP), seeking to maximise expected cumulative reward without direct knowledge of system dynamics or the reward structure's optimum.

Motivation. Classical optimal control methods — such as Linear Quadratic Regulation (LQR) and model predictive control — require explicit knowledge of system dynamics $f(x, u)$ and can be computationally intractable for nonlinear or high-dimensional systems. Policy gradient RL sidesteps these requirements by directly optimising a parameterised policy through gradient ascent on expected return, making it applicable to problems where dynamics are unknown, partially observed, or too complex for analytic treatment.

Continuous action spaces. Many physical control problems — robotics, energy systems, process control — have continuous action spaces. This necessitates policies that output probability distributions over continuous actions, for which Gaussian policies provide a natural, differentiable, and analytically tractable choice.

Goals of this project. This project has four interconnected goals:

1. Formulate control problems as MDPs and derive the policy gradient theorem from first principles.
2. Implement and compare policy gradient algorithms (REINFORCE, A2C, NPG) with rigorous variance analysis.
3. Extend the framework to continuous action spaces using Gaussian policies and stochastic optimisation.
4. Establish the connection between learned policies and the analytic HJB/DARE optimum for LQR systems.

Organisation. Section 2 introduces MDPs and the policy optimisation objective. Section 3 derives the policy gradient theorem and its practical estimators. Section 4 analyses variance and baseline subtraction. Section 5 presents the algorithms implemented. Section 6 develops Gaussian policies for continuous action spaces. Section 7 connects policy gradient RL with stochastic optimal control and the HJB equation. Section 8 presents our experimental results. Section 10 concludes and identifies directions for future work.

2 Markov Decision Processes

Definition 2.1 Markov Decision Process

A *Markov Decision Process* (MDP) is a tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$ where:

- $\mathcal{S} \subseteq \mathbb{R}^n$ is the state space;
- $\mathcal{A} \subseteq \mathbb{R}^m$ is the (continuous) action space;
- $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow \mathbb{R}_{\geq 0}$ is the transition kernel, with $P(s' \mid s, a)$ the density of the next state;
- $R : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the expected reward function;
- $\gamma \in [0, 1)$ is the discount factor.

The Markov property requires that the next state s_{t+1} is conditionally independent of the history $(s_0, a_0, \dots, s_{t-1}, a_{t-1})$ given (s_t, a_t) :

$$P(s_{t+1} \mid s_0, a_0, \dots, s_t, a_t) = P(s_{t+1} \mid s_t, a_t). \quad (1)$$

2.1 Policies and Trajectories

A *stochastic policy* $\pi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}$ is a conditional density over actions given states. A *trajectory* $\tau = (s_0, a_0, s_1, a_1, \dots, s_T, a_T)$ is generated by initialising $s_0 \sim \mu_0$, then iterating:

$$a_t \sim \pi(\cdot \mid s_t), \quad s_{t+1} \sim P(\cdot \mid s_t, a_t). \quad (2)$$

The probability density of a trajectory under π factorises as:

$$p_\pi(\tau) = \mu_0(s_0) \prod_{t=0}^T \pi(a_t \mid s_t) \cdot P(s_{t+1} \mid s_t, a_t). \quad (3)$$

2.2 The Policy Optimisation Objective

We seek a policy π_θ parameterised by $\theta \in \mathbb{R}^d$ that maximises the *expected discounted return*:

$$J(\theta) = \mathbb{E}_{\tau \sim p_{\pi_\theta}} [G(\tau)], \quad G(\tau) = \sum_{t=0}^T \gamma^t r(s_t, a_t). \quad (4)$$

The *state value function* and *action-value function* are:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s \right], \quad (5)$$

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_t \mid s_0 = s, a_0 = a \right]. \quad (6)$$

These satisfy the *Bellman consistency equations*:

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot \mid s)} [Q^\pi(s, a)], \quad (7)$$

$$Q^\pi(s, a) = R(s, a) + \gamma \mathbb{E}_{s' \sim P(\cdot \mid s, a)} [V^\pi(s')]. \quad (8)$$

The *advantage function* captures the relative value of action a vs the expected value of the policy:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s). \quad (9)$$

3 The Policy Gradient Theorem

3.1 Derivation

The key challenge is computing $\nabla_\theta J(\theta)$. The objective depends on θ both through the policy probabilities and through the trajectory distribution. The *log-derivative trick* (also called the REINFORCE or score function trick) resolves this.

Theorem 3.1 (Policy Gradient Theorem (sutton1999))

For a differentiable stochastic policy π_θ , the gradient of the expected return (4) is:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t \mid s_t) \cdot Q^{\pi_\theta}(s_t, a_t) \right]. \quad (10)$$

Proof. We compute $\nabla_\theta J(\theta)$ directly:

$$\begin{aligned}
\nabla_\theta J(\theta) &= \nabla_\theta \int p_{\pi_\theta}(\tau) G(\tau) d\tau \\
&= \int G(\tau) \nabla_\theta p_{\pi_\theta}(\tau) d\tau \\
&= \int G(\tau) p_{\pi_\theta}(\tau) \nabla_\theta \log p_{\pi_\theta}(\tau) d\tau \quad (\text{log-derivative trick: } \nabla \log f = \nabla f / f) \\
&= \mathbb{E}_\tau [G(\tau) \nabla_\theta \log p_{\pi_\theta}(\tau)]. \tag{11}
\end{aligned}$$

Expanding the log-likelihood of the trajectory:

$$\nabla_\theta \log p_{\pi_\theta}(\tau) = \nabla_\theta \log \mu_0(s_0) + \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) + \sum_t \nabla_\theta \log P(s_{t+1} | s_t, a_t). \tag{12}$$

The first and third terms are zero (they do not depend on θ), giving:

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[G(\tau) \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \right]. \tag{13}$$

Using the causality of rewards (rewards at time $t' < t$ cannot be influenced by a_t):

$$\nabla_\theta J(\theta) = \mathbb{E}_\tau \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right], \tag{14}$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ is the return from step t . Since $\mathbb{E}[Q^\pi(s_t, a_t) | s_t, a_t] = Q^\pi(s_t, a_t)$, this yields (10). $\square$

3.2 The Score Function

The vector $\nabla_\theta \log \pi_\theta(a | s)$ is called the *score function* or *likelihood ratio gradient*. It measures the direction in parameter space that increases the log-likelihood of action a given state s . The policy gradient (10) can be interpreted as: *increase the probability of actions that lead to high returns, and decrease the probability of actions that lead to low returns*.

3.3 The REINFORCE Estimator

In practice, $Q^\pi(s_t, a_t)$ is unknown and is estimated by Monte Carlo rollouts. Given a trajectory $\tau^{(i)}$ under π_θ , the **REINFORCE** estimator (williams1992) is:

$$\hat{\nabla}_\theta J(\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}. \tag{15}$$

This estimator is *unbiased*: $\mathbb{E}[\hat{\nabla}_\theta J] = \nabla_\theta J(\theta)$. However, its variance can be extremely high, which motivates the techniques described in Section 4.

4 Variance Reduction

4.1 The Variance Problem

The REINFORCE estimator (15) is unbiased but has high variance because G_t can vary greatly across trajectories even for a fixed policy. High gradient variance leads to slow, noisy learning and is arguably the central practical challenge in policy gradient methods.

To see why, consider the variance of the single-step estimator for a scalar return G :

$$\text{Var} [\nabla \log \pi \cdot G] = \mathbb{E} [(\nabla \log \pi)^2 G^2] - (\mathbb{E} [\nabla \log \pi \cdot G])^2. \tag{16}$$

This grows with the second moment of G , which can be very large.

4.2 Baseline Subtraction

Adding any function $b(s_t)$ that does not depend on a_t to the return does not change the expected gradient:

Proposition 4.1 Zero-Mean Property of Baseline

For any function $b : \mathcal{S} \rightarrow \mathbb{R}$:

$$\mathbb{E}_\pi [\nabla_\theta \log \pi_\theta(a | s) \cdot b(s)] = 0. \quad (17)$$

Proof.

$$\begin{aligned} \mathbb{E}_\pi [\nabla \log \pi(a|s) \cdot b(s)] &= \int \pi(a|s) \nabla \log \pi(a|s) b(s) da \\ &= \int \nabla \pi(a|s) b(s) da \\ &= b(s) \nabla \int \pi(a|s) da = b(s) \nabla 1 = 0. \end{aligned}$$

□

The *baseline-subtracted* estimator:

$$\hat{\nabla}_\theta J = \frac{1}{N} \sum_{i,t} \nabla \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \cdot (G_t^{(i)} - b(s_t^{(i)})) \quad (18)$$

is therefore still unbiased, but its variance is:

$$\text{Var} [\nabla \log \pi \cdot (G - b)] = \text{Var}[\nabla \log \pi \cdot G] - 2 \text{Cov}(\nabla \log \pi \cdot G, \nabla \log \pi \cdot b) + \text{Var}[\nabla \log \pi \cdot b]. \quad (19)$$

The variance is minimised by the optimal baseline:

$$b^*(s) = \frac{\mathbb{E} [\|\nabla \log \pi\|^2 Q^\pi(s, \cdot)]}{\mathbb{E} [\|\nabla \log \pi\|^2]} \approx V^\pi(s). \quad (20)$$

In practice, $b(s) = V^\pi(s)$ (the state value function) is an excellent approximation to b^* and is the standard choice.

4.3 Advantage Estimation

Using $b(s) = V^\pi(s)$ gives the advantage:

$$A^\pi(s_t, a_t) = Q^\pi(s_t, a_t) - V^\pi(s_t). \quad (21)$$

The advantage can be estimated via:

Monte Carlo: $\hat{A}_t^{\text{MC}} = G_t - V_\phi(s_t)$ (high variance, zero bias).

TD(0): $\hat{A}_t^{\text{TD}} = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$ (low variance, biased if $V_\phi \neq V^\pi$).

Generalised Advantage Estimation (GAE) (schulman2015) interpolates between these extremes:

$$\hat{A}_t^{\text{GAE}(\lambda)} = \sum_{\ell=0}^{\infty} (\gamma \lambda)^\ell \delta_{t+\ell}, \quad \delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t), \quad (22)$$

where $\lambda \in [0, 1]$ controls the bias-variance trade-off: $\lambda = 0$ gives TD(0) (low variance), $\lambda = 1$ gives Monte Carlo (zero bias).

5 Algorithms

5.1 REINFORCE

Algorithm 1 REINFORCE with Baseline (**williams1992**)

Require: Policy π_θ , value network V_ϕ , learning rates α_π, α_V , discount γ , episodes per iteration N

- 1: **for** $k = 1, 2, \dots$ **do**
- 2: Collect N trajectories $\{\tau^{(i)}\}_{i=1}^N$ under π_θ
- 3: **for** each trajectory $\tau^{(i)} = (s_0, a_0, r_0, \dots, s_T, a_T, r_T)$ **do**
- 4: Compute returns: $G_t^{(i)} \leftarrow \sum_{k=t}^T \gamma^{k-t} r_k^{(i)}$
- 5: Compute advantages: $\hat{A}_t^{(i)} \leftarrow G_t^{(i)} - V_\phi(s_t^{(i)})$
- 6: **end for**
- 7: Normalise advantages: $\hat{A} \leftarrow (\hat{A} - \mu_{\hat{A}}) / (\sigma_{\hat{A}} + \epsilon)$
- 8: Policy gradient: $g \leftarrow \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_t^{(i)} | s_t^{(i)}) \cdot \hat{A}_t^{(i)}$
- 9: Entropy bonus: $g \leftarrow g + \beta \nabla_\theta H[\pi_\theta(\cdot | s_t^{(i)})]$
- 10: Update policy: $\theta \leftarrow \theta + \alpha_\pi \cdot g$
- 11: Update critic: $\phi \leftarrow \phi - \alpha_V \nabla_\phi \frac{1}{N} \sum_{i,t} (V_\phi(s_t^{(i)}) - G_t^{(i)})^2$
- 12: **end for**

5.2 Advantage Actor-Critic (A2C)

A2C uses GAE (22) advantages and jointly trains actor and critic. The combined loss is:

$$\mathcal{L}(\theta, \phi) = \underbrace{-\mathbb{E} \left[\log \pi_\theta(a|s) \cdot \hat{A}^{\text{GAE}} \right]}_{\text{actor loss}} + \underbrace{c_V \mathbb{E} \left[(V_\phi(s) - G)^2 \right]}_{\text{critic loss}} - \underbrace{c_H \mathbb{E} [H[\pi_\theta(\cdot | s)]]}_{\text{entropy bonus}}, \quad (23)$$

where $c_V = 0.5$ and $c_H = 0.01$ are weighting coefficients.

5.3 Natural Policy Gradient

Standard gradient ascent performs poorly when the policy parameterisation is non-isotropic: a large step in parameter space may correspond to a tiny change in policy distribution, or vice versa.

The *Natural Policy Gradient* (NPG) (**kakade2001**) corrects for this by using the Fisher Information Matrix (FIM) as the Riemannian metric:

$$\mathbf{F}(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a|s) (\nabla_\theta \log \pi_\theta(a|s))^\top \right]. \quad (24)$$

The natural gradient update is:

$$\theta \leftarrow \theta + \alpha \mathbf{F}(\theta)^{-1} \nabla_\theta J(\theta). \quad (25)$$

Since $\mathbf{F}(\theta)^{-1}$ is $O(d^2)$ to compute (where d is the number of parameters), we use the conjugate gradient (CG) algorithm to solve $\mathbf{F}(\theta)x = \nabla_\theta J(\theta)$ for x , exploiting the identity:

$$\mathbf{F}(\theta)v = \nabla_\theta \left[(\nabla_\theta \text{KL}(\pi_\theta \| \pi_\theta^{\text{old}}))^\top v \right]. \quad (26)$$

This requires only $O(d)$ Fisher-vector products via automatic differentiation.

Remark 5.1 Connection to TRPO

Trust Region Policy Optimisation (TRPO) (schulman2015b) solves:

$$\max_{\theta} \mathbb{E} \left[\frac{\pi_{\theta}(a|s)}{\pi_{\theta_{\text{old}}}(a|s)} \hat{A} \right] \quad \text{s.t.} \quad \mathbb{E}[\text{KL}(\pi_{\theta_{\text{old}}} \| \pi_{\theta})] \leq \delta. \quad (27)$$

The first-order approximation of this constrained problem yields exactly the natural gradient update (25).

6 Gaussian Policies for Continuous Action Spaces

6.1 Parameterisation

For continuous action spaces $\mathcal{A} \subseteq \mathbb{R}^m$, we parameterise the policy as a multivariate Gaussian:

$$\pi_{\theta}(a | s) = \mathcal{N} \left(a \mid \mu_{\theta}(s), \text{diag}(\sigma_{\theta}(s))^2 \right), \quad (28)$$

where $\mu_{\theta} : \mathcal{S} \rightarrow \mathbb{R}^m$ and $\sigma_{\theta} : \mathcal{S} \rightarrow \mathbb{R}_+^m$ are neural networks outputting the mean and standard deviation respectively.

The log-probability of action a under this policy is:

$$\log \pi_{\theta}(a | s) = -\frac{1}{2} \sum_{j=1}^m \left(\frac{a_j - \mu_j(s)}{\sigma_j(s)} \right)^2 - \sum_{j=1}^m \log \sigma_j(s) - \frac{m}{2} \log(2\pi). \quad (29)$$

The entropy of the Gaussian policy is:

$$H[\pi_{\theta}(\cdot | s)] = \sum_{j=1}^m \log \sigma_j(s) + \frac{m}{2} (1 + \log(2\pi)). \quad (30)$$

6.2 Score Function for Gaussian Policy

For the Gaussian policy (28), the score function $\nabla_{\theta} \log \pi_{\theta}(a | s)$ decomposes cleanly. For a linear mean $\mu(s) = Ws + b$ with global log σ :

$$\nabla_W \log \pi(a|s) = \frac{(a - \mu(s))}{\sigma^2} \otimes s, \quad (31)$$

$$\nabla_b \log \pi(a|s) = \frac{(a - \mu(s))}{\sigma^2}, \quad (32)$$

$$\nabla_{\log \sigma} \log \pi(a|s) = \frac{(a - \mu(s))^2}{\sigma^2} - 1. \quad (33)$$

For a neural network $\mu_{\theta}(s)$, these gradients propagate through the network via backpropagation.

6.3 Reparameterisation Trick

Sampling $a \sim \pi_{\theta}(\cdot | s)$ can be written as:

$$a = \mu_{\theta}(s) + \sigma_{\theta}(s) \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I). \quad (34)$$

This *reparameterisation* makes the sampling operation differentiable with respect to θ , enabling gradient flow through the sampled action. While not used directly in REINFORCE (which uses the score function), reparameterisation is the foundation of algorithms like SAC (haarnoja2018).

6.4 Network Architecture

Both the actor and critic use multi-layer perceptrons with architecture:

$$s \xrightarrow{\text{Linear}} \mathbb{R}^{64} \xrightarrow{\tanh} \mathbb{R}^{64} \xrightarrow{\tanh} \text{output}.$$

Orthogonal weight initialisation (**saxe2013**) is used throughout, with a smaller scale factor for the output layer to encourage initial exploration.

7 Connection to Stochastic Optimal Control

7.1 The Hamilton-Jacobi-Bellman Equation

Stochastic optimal control seeks a control law $u^*(x, t)$ that minimises a cumulative cost:

$$J(x_0) = \mathbb{E} \left[\int_0^\infty e^{-\rho t} L(x_t, u_t) dt \right]. \quad (35)$$

The *Hamilton-Jacobi-Bellman (HJB) equation* provides a necessary and sufficient condition for optimality:

$$\rho V^*(x) = \min_u \left[L(x, u) + f(x, u)^\top \nabla_x V^*(x) + \frac{1}{2} \text{tr}(\Sigma \nabla_x^2 V^*(x)) \right], \quad (36)$$

where $f(x, u)$ is the drift and Σ is the diffusion covariance.

The optimal policy is recovered from the value function:

$$u^*(x) = u \left[L(x, u) + f(x, u)^\top \nabla_x V^*(x) \right]. \quad (37)$$

7.2 Linear-Quadratic Regulator

For the LQR problem with dynamics $x_{t+1} = Ax_t + Bu_t + w_t$, $w_t \sim \mathcal{N}(0, W)$, and quadratic cost $L(x, u) = x^\top Qx + u^\top Ru$, the HJB value function is quadratic:

$$V^*(x) = -x^\top P^* x + \text{const}, \quad (38)$$

where P^* is the unique positive definite solution to the *Discrete Algebraic Riccati Equation* (DARE):

$$P^* = Q + A^\top P^* A - A^\top P^* B (R + B^\top P^* B)^{-1} B^\top P^* A. \quad (39)$$

The optimal gain matrix and control law are:

$$K^* = (R + B^\top P^* B)^{-1} B^\top P^* A, \quad u^* = -K^* x. \quad (40)$$

7.3 RL as Approximate Dynamic Programming

The Bellman optimality equation in RL:

$$V^*(s) = \max_a [R(s, a) + \gamma \mathbb{E}_{s'} [V^*(s')]] \quad (41)$$

is the discrete-time analogue of the HJB equation (36). The correspondence is:

Optimal Control	Reinforcement Learning
HJB value function $V^*(x)$	Bellman value function $V^*(s)$
Optimal control $u^*(x)$	Optimal policy $\pi^*(a s)$
Riccati equation (DARE)	Bellman equation
Algebraic solution K^*x	Parameterised policy π_θ
Pontryagin maximum principle	Policy gradient theorem

Key insight. In Actor-Critic methods, the critic $V_\phi(s)$ is an *approximation* to the HJB value function $V^*(s)$. After training on the LQR environment, we expect $V_\phi(s) \approx -s^\top P^* s$, which we verify experimentally in Section 8.5.

8 Experiments

8.1 Experimental Setup

All experiments are implemented in Python using PyTorch ([paszke2019](#)) for automatic differentiation and NumPy/SciPy for linear algebra. Environments are implemented from scratch to give full control over system parameters. Random seeds are fixed for reproducibility; multi-seed experiments use seeds 0, 1, 2, 3, 4.

Common hyperparameters:

Hyperparameter	Value	Rationale
Discount factor γ	0.99	Long-horizon tasks
Actor learning rate α_π	3×10^{-4}	Adam, stable for PG
Critic learning rate α_V	10^{-3}	Faster critic convergence
Hidden layer sizes	(64, 64)	Balanced capacity/speed
Activation function	tanh	Bounded, differentiable
Entropy coefficient c_H	0.01	Mild exploration
GAE λ	0.95	Near-MC, low bias
Gradient clip norm	0.5	Prevent exploding grads

8.2 Experiment 1: LQR Policy Gradient vs Analytic Optimum

Setup. We train five policy gradient variants on a 4-dimensional LQR system with 2-dimensional control input, comparing against the analytic optimal gain K^* obtained from the DARE solution.

Results. Table 1 summarises the final evaluation performance after 200 training iterations with 10 episodes per iteration.

Table 1: Exp 1: LQR Performance Comparison (mean $\pm$ std, 50 evaluation episodes)

Agent	Mean Reward	Std	Regret (%)
Optimal LQR (K^*)	-312.4	18.6	0.0
Natural PG	-319.7	24.1	2.3
A2C (GAE, $\lambda = 0.95$)	-323.5	27.3	3.6
REINFORCE (NN baseline)	-331.2	35.4	6.0
REINFORCE (mean baseline)	-348.6	58.2	11.6
REINFORCE (no baseline)	-410.3	112.4	31.3
Random Policy	-1243.2	285.7	297.8

Key observations:

- Natural PG achieves the lowest regret (2.3%) due to its parameter-space invariance and faster convergence in the direction of steepest ascent.
- A2C’s GAE advantage estimation significantly outperforms vanilla REINFORCE.
- Without a baseline, REINFORCE has much higher variance (std = 112.4 vs 35.4 for NN baseline), consistent with the theory in Section 4.
- The linear PG agent (REINFORCE with linear $\mu(s) = Ws + b$) learns a gain matrix converging toward K^* , confirming the theory.

8.3 Experiment 2: Continuous Pendulum Swing-Up

Setup. The continuous pendulum environment has state $s = [\cos \theta, \sin \theta, \dot{\theta}] \in \mathbb{R}^3$ and continuous torque action $\tau \in [-2, 2]$. The reward is:

$$r(\theta, \dot{\theta}, \tau) = - \left[(\theta - \pi)^2 + 0.1\dot{\theta}^2 + 0.001\tau^2 \right], \quad (42)$$

penalising deviation from the upright position $\theta = 0$. No analytic solution exists for this nonlinear system.

Results.

Table 2: Exp 2: Pendulum Performance (mean $\pm$ std, 50 evaluation episodes)

Agent	Mean Reward	Std
A2C (GAE, $\lambda = 0.95$)	-152.3	41.2
Natural PG	-164.7	55.6
REINFORCE (NN baseline)	-198.4	87.3
PID Controller	-245.1	62.0
Random Policy	-1421.8	193.4

A2C outperforms the hand-tuned PID controller, demonstrating that learned Gaussian policies can discover effective control strategies without system dynamics knowledge. The PID controller struggles because the swing-up phase requires non-linear energy injection, while PID linearises around equilibrium.

8.4 Experiment 3: Variance Reduction Study

Setup. On the Double Integrator environment, we measure the gradient variance and signal-to-noise ratio for three baseline configurations over 5 seeds.

Results.

Table 3: Exp 3: Gradient Variance Comparison (50 gradient samples)

Baseline	Mean $\ g\ $	Gradient Var	SNR
No baseline	1.24	3.87	0.63
Mean return	1.19	2.31	1.02
NN value function	1.17	1.56	1.48

The NN value baseline reduces gradient variance by 59.7% compared to no baseline, while maintaining an unbiased estimate of the true gradient. The SNR increases by a factor of $\sim 2.3\times$, explaining the significantly faster and more stable convergence observed with baselines.

8.5 Experiment 4: HJB Connection

Setup. On a 2D LQR system (chosen for visualisation), we track critic convergence toward the analytic HJB value $V^*(s) = -s^\top P^* s$ throughout training.

Results.

After 250 iterations, the critic achieves a Pearson correlation of 0.98 with the analytic HJB solution and relative error below 5%. Value function contour plots (see Figure ??) show that the learned V_ϕ accurately captures the quadratic shape of $V^*(s) = -s^\top P^* s$.

Gain analysis shows that the learned neural policy, linearised around the origin, has a gain matrix K_{eff} with Frobenius distance 0.043 from the optimal K^* , confirming that the RL policy is discovering the same structure as the LQR optimum.

Table 4: Exp 4: Critic Approximation Quality vs Analytic HJB Solution

Training Iter	MAE	RMSE	Rel. Error	Correlation
10	42.3	58.7	0.81	0.34
50	18.6	26.4	0.38	0.72
100	8.9	12.1	0.17	0.88
150	5.2	7.3	0.10	0.94
200	3.8	5.1	0.07	0.97
250	2.9	4.0	0.05	0.98

9 Discussion

9.1 Comparative Analysis of Algorithms

REINFORCE. Simple to implement and theoretically principled. The primary weakness is high gradient variance due to Monte Carlo return estimation. Variance is dramatically reduced by adding a learned value baseline, but convergence remains slower than A2C.

A2C. GAE provides an excellent bias-variance trade-off with $\lambda = 0.95$. Joint training of actor and critic is efficient but requires careful hyperparameter tuning of c_V and c_H . A2C consistently outperforms REINFORCE across our experiments.

Natural Policy Gradient. The FIM-weighted update is theoretically superior (policy-space invariance, monotonic improvement guarantees) and achieves the lowest regret on LQR. The main limitation is the $O(d^2)$ naive complexity of FIM computation, mitigated by CG but still slower per iteration than A2C.

9.2 Gaussian Policies and Exploration

Gaussian policies provide a natural exploration mechanism: the standard deviation $\sigma(s)$ directly controls exploration. We observed that the entropy coefficient c_H is important for performance: too low and the policy collapses prematurely to a suboptimal deterministic policy; too high and learning is destabilised.

The state-independent log-std parameterisation (global σ) is generally more stable than state-dependent std, though the latter can express richer exploration strategies.

9.3 RL vs Classical Control

For LQR problems where the DARE solution is tractable, classical optimal control is orders of magnitude more sample-efficient. However, policy gradient RL becomes advantageous when:

1. System dynamics are unknown or stochastic in complex ways;
2. The state/action space is too large for analytic treatment;
3. The reward structure is learned or non-convex.

Our experiments suggest that policy gradient RL can recover near-optimal control strategies for LQR systems, closing roughly 95% of the gap to the analytic optimum within 200 training iterations.

10 Conclusion

We have developed a comprehensive framework for policy gradient reinforcement learning applied to continuous control problems. Starting from MDP fundamentals, we derived the policy gradient

theorem, implemented Gaussian policies with neural network parameterisation, and studied variance reduction through baseline subtraction both theoretically and empirically.

Our key findings are:

1. **Baseline subtraction is essential.** Neural network value baselines reduce gradient variance by $\sim 60\%$, directly improving convergence speed and stability.
2. **GAE provides an excellent bias-variance trade-off.** A2C with $\lambda = 0.95$ consistently outperforms Monte Carlo REINFORCE while remaining computationally efficient.
3. **Natural gradients improve convergence.** NPG achieves the lowest regret ($< 2.5\%$) on LQR by exploiting the Fisher information geometry of the policy space.
4. **Learned critics converge to the HJB solution.** After sufficient training, critic correlations of > 0.97 with the analytic $V^*(s) = -s^\top P^* s$ demonstrate that RL is effectively solving the Bellman equation, recovering the same mathematical object as optimal control.
5. **Gaussian policies generalise beyond linear systems.** On the nonlinear pendulum swing-up task, learned Gaussian policies outperform hand-tuned PID controllers without access to system dynamics.

Future work. Promising directions include: Proximal Policy Optimisation (PPO) (schulman2017) for more stable updates; model-based policy gradient methods that combine learned dynamics with gradient estimation; and multi-task policy gradient for learning transferable control strategies.

A Proof of Variance Reduction Bound

Let $g = \nabla \log \pi(a|s) \cdot (G - b)$ be the baseline-subtracted gradient estimate. The variance reduction from baseline b is:

$$\begin{aligned} \text{Var}[g_{\text{no base}}] - \text{Var}[g_b] &= 2 \text{Cov}(\nabla \log \pi \cdot G, \nabla \log \pi \cdot b) - \text{Var}[\nabla \log \pi \cdot b] \\ &= 2\mathbb{E}[(\nabla \log \pi)^2 G b] - \mathbb{E}[(\nabla \log \pi)^2 b^2]. \end{aligned} \quad (43)$$

Maximising over b gives $b^*(s) = \mathbb{E}[(\nabla \log \pi)^2 G] / \mathbb{E}[(\nabla \log \pi)^2]$. For the Gaussian policy with unit Fisher norm (i.e., $\|\nabla \log \pi\|^2 = 1$), this simplifies to $b^*(s) = \mathbb{E}[G|s] = V^\pi(s)$.

B Discrete Algebraic Riccati Equation

The DARE (39) arises from minimising the infinite-horizon LQR cost. Define the cost-to-go $V_k(x) = \min_u \sum_{t=0}^k (x_t^\top Q x_t + u_t^\top R u_t)$. The value iteration:

$$V_{k+1}(x) = x^\top Q x + \min_u [u^\top R u + V_k(Ax + Bu)] \quad (44)$$

has the closed-form minimiser $u^* = -(R + B^\top P_k B)^{-1} B^\top P_k A x$, giving the Riccati recursion:

$$P_{k+1} = Q + A^\top P_k A - A^\top P_k B (R + B^\top P_k B)^{-1} B^\top P_k A. \quad (45)$$

Under stabilisability and detectability conditions, $P_k \rightarrow P^*$ as $k \rightarrow \infty$, and P^* is the unique PSD solution to the DARE.

C Experimental Hyperparameters

Table 5: Complete hyperparameter table for all experiments

Hyperparameter	Exp 1	Exp 2	Exp 3	Exp 4
State dimension	4	3	2	2
Action dimension	2	1	1	1
Training iterations	200	300	150	250
Episodes/iteration	10	10	10	15
Discount γ	0.99	0.99	0.99	0.99
Actor LR	3e-4	3e-4	3e-4	3e-4
Critic LR	1e-3	1e-3	1e-3	1e-3
Hidden sizes	(64,64)	(64,64)	(32,32)	(64,64)
Entropy coef	0.005	0.01	0.005	0.005
GAE λ	0.95	0.95	—	—
CG iters (NPG)	10	10	—	—
NPG damping	0.01	0.01	—	—